MODELADO ESTRUCTURAL

Ferney Eduardo Ballesta Avila
20231020141

Mariam Betin Escobar
20232020300

Universidad Distrital Francisco José de Caldas
Facultad de Ingeniería
Ingeniería de Sistemas
Bogotá D.C.
20 de abril de 2026

MODELADO ESTRUCTURAL

Ferney Eduardo Ballesta Avila
20231020141

Mariam Betin Escobar
20232020300

Reporte técnico

Presentado a:
Henry Alberto Diosa

Universidad Distrital Francisco José de Caldas
Facultad de Ingeniería
Ingeniería de Sistemas
Bogotá D.C.
20 de abril de 2026

CUADRO DE REVISIONES

Versión	Fecha	Autor	Descripción
1.0	08/04/2026	Equipo de trabajo	Versión inicial
1.1	08/04/2026	Equipo de trabajo	Se añadieron los bocetos actualizados con respecto al entregable de especificaciones funcionales
2.0	13/04/2026	Equipo de trabajo	Se añadieron algunos diccionarios de clase, principalmente los correspondientes a la capa de datos
3.0	17/04/2026	Equipo de trabajo	Se realizaron correcciones con respecto a las relaciones en el diagrama de clases, y en el tipo de relacion que tienen con respecto a los mapeadores
3.1	17/04/2026	Equipo de trabajo	Se completo el diagrama de clases.
3.2	17/04/2026	Equipo de trabajo	Integracion de modulos y revision de consistencia entre la logica y la base de datos (el diagrama).
4.0	19/04/2026	Equipo de trabajo	Se creo el mapa de navegacion, basado en los bocetos de IGU, y se completaron los diccionarios de clases.
5.0	20/04/2026	Equipo de trabajo	Versión final con ajustes generales y correcciones.

CONTENIDO

pág.

1.. INTRODUCCIÓN 1

2.. DIAGRAMA DE CLASES 2

3.. DICCIONARIO DE CLASES 4

4.. MODELADO DE PERSISTENCIA 30
4.1. Patrones de Fuente de Datos 30
4.2. Estrategia de Mapeo 31
4.2.1. Operaciones CRUD en los Mappers 32
4.3. Modelo Relacional 33

5.. DISEÑO BASICO DE IGU 53
5.1. Bocetos visuales IGU 53
5.1.1. Acceso 53
5.1.2. Formulario de inscripción de adultos mayores 53
5.1.3. Cierre de sesión 55
5.1.4. Administrador de usuarios 55
5.1.5. Administrador de Hogares 57
5.1.6. Administrador de Residentes 59
5.1.7. Formularios de seguimiento y atención al residente 62
5.1.8. Panel del Trabajador Social 64
5.1.9. Panel del Personal de Enfermería 65
5.1.10. Panel del Coordinador 67
5.1.11. Panel del Responsable del Residente 70
5.1.12. Registros de seguimiento 71
5.2. Mapa de navegación 71

6.. ULTIMA PROPUESTA DE NEGOCIACIÓN 73

7.. REFERENCIAS 74

1. INTRODUCCIÓN

El presente documento tiene como propósito desarrollar el modelado estructural de un sistema orientado a la gestión de información en un contexto gerontológico, enfocado en la administración de datos relacionados con adultos mayores, sus planes de atención y su entorno institucional. Este modelado se realiza haciendo uso de principios de diseño orientado a objetos y buenas prácticas de ingeniería de software, con el fin de establecer una representación clara, coherente y estructurada del sistema que facilite su comprensión, diseño e implementación.

En primer lugar, se presenta el diagrama de clases, el cual constituye una visión general del sistema desde una perspectiva estática, permitiendo comprender la organización de las entidades, sus atributos y métodos, así como las interacciones que se establecen entre ellas. Posteriormente, se desarrolla el diccionario de clases, donde se detalla de forma precisa cada uno de los elementos definidos en el modelo, proporcionando una descripción más profunda que elimina ambigüedades y facilita su interpretación.

Adicionalmente, se incluye el modelado de persistencia, en el cual se define la forma en que la información del sistema será almacenada y gestionada, haciendo uso de patrones de diseño que favorecen la separación de responsabilidades y la mantenibilidad del software. Finalmente, se presenta una propuesta básica de interfaz gráfica de usuario, con el objetivo de ofrecer una aproximación a la interacción del usuario con el sistema.

En conjunto, este documento busca no solo describir la estructura del sistema, sino también establecer una base sólida para su futura implementación, asegurando coherencia, claridad y consistencia en cada uno de sus componentes.

2. DIAGRAMA DE CLASES

NOTA: Los diagramas pueden verse reducidos. Para una mejor visualización, se recomienda al lector revisar la versión digital y hacer zoom.

El diagrama de clases presenta la estructura estática del sistema, mostrando las entidades principales de la capa de datos y del subsistema de control de acceso, así como sus atributos, métodos y relaciones. En este diagrama se incluyen las clases persistentes del dominio, tales como AdultoMayor, PlanMedico, DiagnosticoPsicosocial, Bitacora y Hogar, junto con sus asociaciones y agregaciones.

Asimismo, se modela el subsistema de seguridad mediante las clases Usuario, UsuarioAcceso, Rol y Permiso, el cual implementa un esquema de control de acceso basado en roles. Las relaciones entre clases reflejan la estructura del dominio, incluyendo asociaciones, agregaciones y composiciones, así como las multiplicidades correspondientes. Esto permite representar de manera clara cómo interactúan las entidades dentro del sistema.

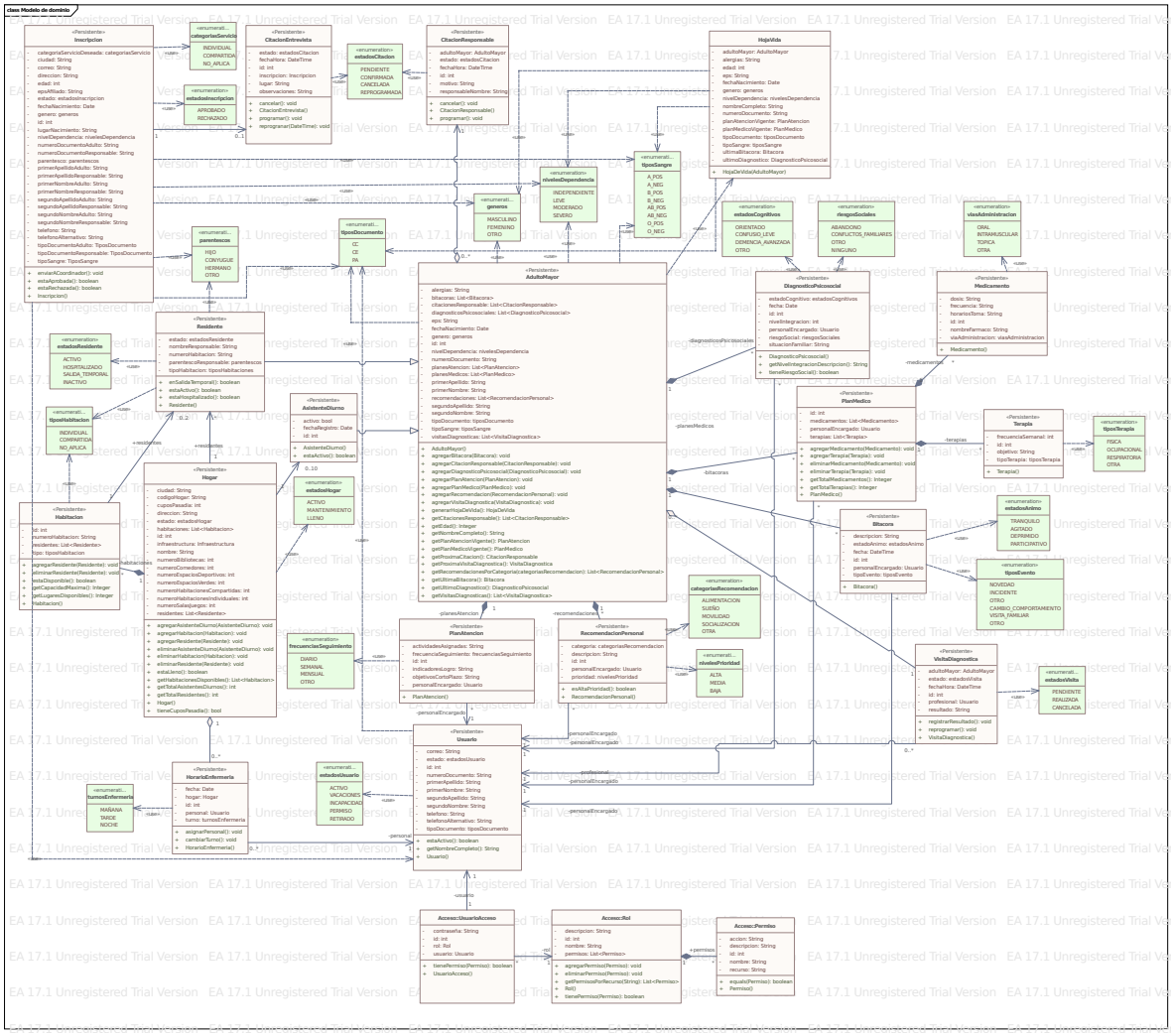


Figura 2.1: Modelo de dominio del HogarViejitoCanoso

3. DICCIONARIO DE CLASES

El diccionario de clases describe de manera exhaustiva cada una de las entidades identificadas en el diagrama de clases del sistema. En este apartado se detallan los atributos y métodos de cada clase, especificando sus tipos de datos, niveles de visibilidad y dominios de valores correspondientes para asegurar una implementación técnica precisa.

Permiso				
Descripción: Representa un permiso de acceso que define una acción permitida sobre un recurso.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	accion	String	Texto libre	
-	descripcion	String	Texto libre	
-	id	int	Entero positivo	
-	nombre	String	Texto libre	
-	recurso	String	Texto libre	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Permiso()	-	void	Constructor de la clase
+	equals	(:Permiso)	boolean	Compara si dos permisos son iguales
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Rol				
Descripción: Representa un rol de usuario en el sistema, agrupando un conjunto de permisos.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	descripcion	String	Texto libre	
-	id	int	Entero positivo	
-	nombre	String	Texto libre	
-	permisos	List<Permiso>	Lista de objetos Permiso	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Rol()	-	void	Constructor de la clase
+	agregarPermiso	(:Permiso)	void	Agrega un permiso al rol
+	eliminarPermiso	(:Permiso)	void	Elimina un permiso del rol
+	getPermisosPorRecurso	(:String)	List<Permiso>	Retorna permisos filtrados por recurso
+	tienePermiso	(:Permiso)	boolean	Verifica si el rol tiene un permiso dado
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

UsuarioAcceso				
Descripción: Gestiona las credenciales de acceso al sistema, vinculando un Usuario con un Rol.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	contraseña	String	Texto cifrado	
-	id	int	Entero positivo	
-	rol	Rol	Objeto de tipo Rol	
-	usuario	Usuario	Objeto de tipo Usuario	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	UsuarioAcceso()	-	void	Constructor de la clase
+	tienePermiso	(:Permiso)	boolean	Verifica si el usuario posee un permiso
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Usuario (Persistente)				
Descripción: Representa al personal del hogar que opera en el sistema. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	correo	String	Texto libre (email)	
-	estado	estadosUsuario	ACTIVO, VACACIONES, INCAPACIDAD, PERMISO, RETIRADO	
-	id	int	Entero positivo	
-	numeroDocumento	String	Texto numérico	
-	primerApellido	String	Texto libre	
-	primerNombre	String	Texto libre	
-	segundoApellido	String	Texto libre	
-	segundoNombre	String	Texto libre	
-	telefono	String	Texto numérico	
-	telefonoAlternativo	String	Texto numérico	
-	tipoDocumento	tiposDocumento	CC, CE, PA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Usuario()	-	void	Constructor de la clase
+	estaActivo	()	boolean	Verifica si el usuario está activo
+	getNombreCompleto	()	String	Retorna el nombre completo concatenado
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Residente (Persistente)				
Descripción: Representa a una persona que reside en el hogar geriátrico. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	estado	estadosResidente	ACTIVO, HOSPITALIZADO, SALIDA_TEMPORAL, INACTIVO	
-	nombreResponsable	String	Texto libre	
-	numeroHabitacion	String	Texto alfanumérico	
-	parentesco Responsable	parentescos	HIJO, CONYUGUE, HERMANO, OTRO	
-	tipoHabitacion	tiposHabitaciones	INDIVIDUAL, COMPARTIDA, NO_APLICA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Residente()	-	void	Constructor de la clase
+	enSalidaTemporal	()	boolean	Indica si está en salida temporal
+	estaActivo	()	boolean	Indica si el residente está activo
+	estaHospitalizado	()	boolean	Indica si está hospitalizado
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Hogar (Persistente)				
Descripción: Representa un hogar geriátrico con su infraestructura, habitaciones y residentes. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	ciudad	String	Texto libre	
-	codigoHogar	String	Texto alfanumérico	
-	cuposPasadia	int	Entero no negativo	
-	direccion	String	Texto libre	
-	estado	estadosHogar	ACTIVO, MANTENIMIENTO, LLENO	
-	habitaciones	List<Habitacion>	Lista de objetos Habitacion	
-	id	int	Entero positivo	
-	infraestructura	Infraestructura	Objeto Infraestructura	
-	nombre	String	Texto libre	
-	numeroBibliotecas	int	Entero no negativo	
-	numeroComedores	int	Entero no negativo	
-	numeroEspacios Deportivos	int	Entero no negativo	
-	numeroEspacios Verdes	int	Entero no negativo	
-	numeroHabitaciones Compartidas	int	Entero no negativo	
-	numeroHabitaciones Individuales	int	Entero no negativo	
-	numeroSalasJuegos	int	Entero no negativo	
-	residentes	List<Residente>	Lista de objetos Residente	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Hogar()	-	void	Constructor de la clase
+	agregarAsistente Diurno	(:AsistenteDiurno)	void	Agrega un asistente diurno
+	agregarHabitacion	(:Habitacion)	void	Agrega una habitación al hogar
+	agregarResidente	(:Residente)	void	Agrega un residente al hogar
+	eliminarAsistente Diurno	(:AsistenteDiurno)	void	Elimina un asistente diurno

Hogar (Persistente)				
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	eliminarHabitacion	(:Habitacion)	void	Elimina una habitación
+	eliminarResidente	(:Residente)	void	Elimina un residente
+	estaLleno	()	boolean	Indica si el hogar está lleno
+	getHabitaciones Disponibles	()	List<Habitacion>	Retorna habitaciones con cupo
+	getTotalAsistentes Diurnos	()	int	Retorna total de asistentes diurnos
+	getTotalResidentes	()	int	Retorna el total de residentes
+	tieneCuposPasadia	()	bool	Indica si hay cupos de pasa día
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Inscripcion (Persistente)			
Descripción: Representa la solicitud de ingreso de un adulto mayor al hogar geriátrico. Clase persistente.			
Capa de atributos			
Vis	**Nombre**	**Tipo**	**Dominio de valores**
-	categoriaServicio Deseada	categoriasServicio	INDIVIDUAL, COMPARTIDA, NO_APLICA
-	ciudad	String	Texto libre
-	correo	String	Texto libre (email)
-	direccion	String	Texto libre
-	edad	int	Entero positivo
-	epsAfiliado	String	Texto libre
-	estado	estadosInscripcion	APROBADO, RECHAZADO
-	fechaNacimiento	Date	Fecha válida
-	genero	generos	MASCULINO, FEMENINO, OTRO
-	id	int	Entero positivo
-	lugarNacimiento	String	Texto libre
-	nivelDependencia	nivelesDependencia	INDEPENDIENTE, LEVE, MODERADO, SEVERO
-	numeroDocumento Adulto	String	Texto numérico
-	numeroDocumento Responsable	String	Texto numérico
-	parentesco	parentescos	HIJO, CONYUGUE, HERMANO, OTRO
-	primerApellido Adulto	String	Texto libre
-	primerApellido Responsable	String	Texto libre
-	primerNombre Adulto	String	Texto libre
-	primerNombre Responsable	String	Texto libre
-	segundoApellido Adulto	String	Texto libre
-	segundoApellido Responsable	String	Texto libre
-	segundoNombre Adulto	String	Texto libre

Inscripcion (Persistente)				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	segundoNombre Responsable	String	Texto libre	
-	telefono	String	Texto numérico	
-	telefonoAlternativo	String	Texto numérico	
-	tipoDocumento Adulto	TiposDocumento	CC, CE, PA	
-	tipoDocumento Responsable	TiposDocumento	CC, CE, PA	
-	tipoSangre	TiposSangre	A_POS, A_NEG, B_POS, B_NEG, AB_POS, AB_NEG, O_POS, O_NEG	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Inscripcion()	-	void	Constructor de la clase
+	enviarACoordinador	()	void	Envía la inscripción al coordinador
+	estaAprobada	()	boolean	Indica si la inscripción fue aprobada
+	estaRechazada	()	boolean	Indica si la inscripción fue rechazada
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

AdultoMayor (Persistente)			
Descripción: Representa al adulto mayor atendido en el hogar, con todos sus registros clínicos y de bienestar. Clase persistente.			
Capa de atributos			
Vis	**Nombre**	**Tipo**	**Dominio de valores**
-	alergias	String	Texto libre
-	bitacoras	List<Bitacora>	Lista de objetos Bitacora
-	citaciones Responsable	List<Citacion Responsable>	Lista de objetos CitacionResponsable
-	diagnosticos Psicosociales	List<Diagnostico Psicosocial>	Lista de objetos DiagnosticoPsicosocial
-	eps	String	Texto libre
-	fechaNacimiento	Date	Fecha válida
-	genero	generos	MASCULINO, FEMENINO, OTRO
-	id	int	Entero positivo
-	nivelDependencia	nivelesDependencia	INDEPENDIENTE, LEVE, MODERADO, SEVERO
-	numeroDocumento	String	Texto numérico
-	planesAtencion	List<PlanAtencion>	Lista de objetos PlanAtencion
-	planesMedicos	List<PlanMedico>	Lista de objetos PlanMedico
-	primerApellido	String	Texto libre
-	primerNombre	String	Texto libre
-	recomendaciones	List<Recomendacion Personal>	Lista de objetos RecomendacionPersonal
-	segundoApellido	String	Texto libre
-	segundoNombre	String	Texto libre
-	tipoDocumento	tiposDocumento	CC, CE, PA
-	tipoSangre	tiposSangre	A_POS, A_NEG, B_POS, B_NEG, AB_POS, AB_NEG, O_POS, O_NEG
-	visitasDiagnosticas	List<Visita Diagnostica>	Lista de objetos VisitaDiagnostica

AdultoMayor (Persistente)				
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	AdultoMayor()	-	void	Constructor de la clase
+	agregarBitacora	(:Bitacora)	void	Agrega una bitácora al historial
+	agregarCitacion Responsable	(:Citacion Responsable)	void	Registra una citación al responsable
+	agregarDiagnostico Psicosocial	(:Diagnostico Psicosocial)	void	Registra un diagnóstico psicosocial
+	agregarPlanAtencion	(:PlanAtencion)	void	Agrega un plan de atención
+	agregarPlanMedico	(:PlanMedico)	void	Agrega un plan médico
+	agregarRecomendacion	(:Recomendacion Personal)	void	Agrega una recomendación personal
+	agregarVisita Diagnostica	(:Visita Diagnostica)	void	Registra una visita diagnóstica
+	generarHojaDeVida	()	HojaDeVida	Genera la hoja de vida del adulto mayor
+	getCitaciones Responsable	()	List<Citacion Responsable>	Retorna la lista de citaciones
+	getEdad	()	Integer	Calcula y retorna la edad
+	getNombreCompleto	()	String	Retorna el nombre completo
+	getPlanAtencion Vigente	()	PlanAtencion	Retorna el plan de atención vigente
+	getPlanMedicoVigente	()	PlanMedico	Retorna el plan médico vigente

AdultoMayor (Persistente)				
Capa de métodos				
+	getProximaCitacion	()	Citacion Responsable	Retorna la próxima citación
+	getProximaVisita Diagnostica	()	Visita Diagnostica	Retorna la próxima visita diagnóstica
+	getRecomendaciones PorCategoria	(:categorias Recomendacion)	List<Recomendacion Personal>	Filtra recomendaciones por categoría
+	getUltimaBitacora	()	Bitacora	Retorna la última bitácora
+	getUltimoDiagnostico	()	Diagnostico Psicosocial	Retorna el último diagnóstico
+	getVisitas Diagnosticas	()	List<Visita Diagnostica>	Retorna todas las visitas diagnósticas
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

AsistenteDiurno (Persistente)				
Descripción: Representa al personal de asistencia diurna asignado al hogar geriátrico. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	activo	bool	true / false	
-	fechaRegistro	Date	Fecha válida	
-	id	int	Entero positivo	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	AsistenteDiurno()	-	void	Constructor de la clase
+	estaActivo	()	boolean	Indica si el asistente está activo
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

RecomendacionPersonal (Persistente)				
Descripción: Registra una recomendación personalizada emitida por el personal para un adulto mayor. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	categoria	categorias Recomendacion	ALIMENTACION, SUEÑO, MOVILIDAD, SOCIALIZACION, OTRA	
-	descripcion	String	Texto libre	
-	id	int	Entero positivo	
-	personalEncargado	Usuario	Objeto de tipo Usuario	
-	prioridad	nivelesPrioridad	ALTA, MEDIA, BAJA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	RecomendacionPersonal	()	void	Constructor de la clase
+	esAltaPrioridad	()	boolean	Indica si la recomendación es de alta prioridad
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Bitacora (Persistente)				
Descripción: Registra un evento o novedad relacionada con un adulto mayor en fecha y hora determinada. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	descripcion	String	Texto libre	
-	estadoAnimo	estadosAnimo	TRANQUILO, AGITADO, DEPRIMIDO, PARTICIPATIVO	
-	fecha	DateTime	Fecha y hora válidas	
-	id	int	Entero positivo	
-	personalEncargado	Usuario	Objeto de tipo Usuario	
-	tipoEvento	tiposEvento	NOVEDAD, INCIDENTE, CAMBIO_COMPORTAMIENTO, VISITA_FAMILIAR, OTRO	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Bitacora()	-	void	Constructor de la clase
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

PlanMedico (Persistente)				
Descripción: Agrupa los medicamentos y terapias prescritas a un adulto mayor. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	id	int	Entero positivo	
-	medicamentos	List<Medicamento>	Lista de objetos Medicamento	
-	personalEncargado	Usuario	Objeto de tipo Usuario	
-	terapias	List<Terapia>	Lista de objetos Terapia	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	PlanMedico()	-	void	Constructor de la clase
+	agregarMedicamento	(:Medicamento)	void	Agrega un medicamento al plan
+	agregarTerapia	(:Terapia)	void	Agrega una terapia al plan
+	eliminarMedicamento	(:Medicamento)	void	Elimina un medicamento del plan
+	eliminarTerapia	(:Terapia)	void	Elimina una terapia del plan
+	getTotalMedicamentos	()	Integer	Retorna el total de medicamentos
+	getTotalTerapias	()	Integer	Retorna el total de terapias
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

PlanAtencion (Persistente)				
Descripción: Define actividades, objetivos e indicadores de seguimiento para la atención integral. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	actividadesAsignadas	String	Texto libre	
-	frecuencia Seguimiento	frecuencias Seguimiento	DIARIO, SEMANAL, MENSUAL, OTRO	
-	id	int	Entero positivo	
-	indicadoresLogro	String	Texto libre	
-	objetivosCortoPlazo	String	Texto libre	
-	personalEncargado	Usuario	Objeto de tipo Usuario	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	PlanAtencion()	-	void	Constructor de la clase
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

DiagnosticoPsicosocial (Persistente)				
Descripción: Registra la evaluación psicosocial de un adulto mayor en una fecha determinada. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	estadoCognitivo	estadosCognitivos	ORIENTADO, CONFUSION_LEVE, DEMENCIA_AVANZADA, OTRO	
-	fecha	Date	Fecha válida	
-	id	int	Entero positivo	
-	nivelIntegracion	int	Entero (escala definida)	
-	personalEncargado	Usuario	Objeto de tipo Usuario	
-	riesgoSocial	riesgosSociales	ABANDONO, CONFLICTOS_FAMILIARES, OTRO, NINGUNO	
-	situacionFamiliar	String	Texto libre	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	DiagnosticoPsicosocial()	-	void	Constructor de la clase
+	getNivelIntegracion Descripcion	()	String	Retorna descripción del nivel de integración
+	tieneRiesgoSocial	()	boolean	Indica si hay riesgo social
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Medicamento (Persistente)				
Descripción: Representa un fármaco prescrito en un plan médico, con su dosis y vía de administración. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	dosis	String	Texto libre	
-	frecuencia	String	Texto libre	
-	horariosToma	String	Texto libre	
-	id	int	Entero positivo	
-	nombreFarmaco	String	Texto libre	
-	viaAdministracion	viasAdministracion	ORAL, INTRAMUSCULAR, TOPICA, OTRA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Medicamento()	-	void	Constructor de la clase
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Terapia (Persistente)				
Descripción: Representa una terapia prescrita en un plan médico, con su tipo y frecuencia semanal. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	frecuenciaSemanal	int	Entero positivo	
-	id	int	Entero positivo	
-	objetivo	String	Texto libre	
-	tipoTerapia	tiposTerapia	FISICA, OCUPACIONAL, RESPIRATORIA, OTRA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Terapia()	-	void	Constructor de la clase
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

Habitacion (Persistente)				
Descripción: Representa una habitación del hogar, con sus residentes asignados y tipo. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	id	int	Entero positivo	
-	numeroHabitacion	String	Texto alfanumérico	
-	residentes	List<Residente>	Lista de objetos Residente (máx. 2)	
-	tipo	tiposHabitacion	INDIVIDUAL, COMPARTIDA, NO_APLICA	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	Habitacion()	-	void	Constructor de la clase
+	agregarResidente	(:Residente)	void	Agrega un residente a la habitación
+	eliminarResidente	(:Residente)	void	Elimina un residente de la habitación
+	estaDisponible	()	boolean	Indica si la habitación tiene cupo
+	getCapacidadMaxima	()	Integer	Retorna la capacidad máxima
+	getLugaresDisponibles	()	Integer	Retorna los lugares disponibles
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

HojaVida				
Descripción: Objeto de valor que consolida la información clínica y personal vigente de un adulto mayor.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	adultoMayor	AdultoMayor	Objeto de tipo AdultoMayor	
-	alergias	String	Texto libre	
-	edad	int	Entero positivo	
-	eps	String	Texto libre	
-	fechaNacimiento	Date	Fecha válida	
-	genero	generos	MASCULINO, FEMENINO, OTRO	
-	nivelDependencia	nivelesDependencia	INDEPENDIENTE, LEVE, MODERADO, SEVERO	
-	nombreCompleto	String	Texto libre	
-	numeroDocumento	String	Texto numérico	
-	planAtencionVigente	PlanAtencion	Objeto de tipo PlanAtencion	
-	planMedicoVigente	PlanMedico	Objeto de tipo PlanMedico	
-	tipoDocumento	tiposDocumento	CC, CE, PA	
-	tipoSangre	tiposSangre	A_POS, A_NEG, B_POS, B_NEG, AB_POS, AB_NEG, O_POS, O_NEG	
-	ultimaBitacora	Bitacora	Objeto de tipo Bitacora	
-	ultimoDiagnostico	DiagnosticoPsicosocial	Objeto de tipo DiagnosticoPsicosocial	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	HojaDeVida	(:AdultoMayor)	void	Constructor que inicializa desde un AdultoMayor
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

CitacionEntrevista (Persistente)				
Descripción: Representa la citación a entrevista vinculada a una inscripción de adulto mayor. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	estado	estadosCitacion	PENDIENTE, CONFIRMADA, CANCELADA, REPROGRAMADA	
-	fechaHora	DateTime	Fecha y hora válidas	
-	id	int	Entero positivo	
-	inscripcion	Inscripcion	Objeto de tipo Inscripcion	
-	lugar	String	Texto libre	
-	observaciones	String	Texto libre	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	CitacionEntrevista()	-	void	Constructor de la clase
+	cancelar	()	void	Cambia el estado a CANCELADA
+	programar	()	void	Registra y confirma la citación
+	reprogramar	(:DateTime)	void	Cambia fecha y marca como REPROGRAMADA
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

CitacionResponsable (Persistente)				
Descripción: Representa una citación al responsable de un adulto mayor para tratar asuntos de seguimiento. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	adultoMayor	AdultoMayor	Objeto de tipo AdultoMayor	
-	estado	estadosCitacion	PENDIENTE, CONFIRMADA, CANCELADA, REPROGRAMADA	
-	fechaHora	DateTime	Fecha y hora válidas	
-	id	int	Entero positivo	
-	motivo	String	Texto libre	
-	responsableNombre	String	Texto libre	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	CitacionResponsable()	-	void	Constructor de la clase
+	cancelar	()	void	Cambia el estado a CANCELADA
+	programar	()	void	Registra y confirma la citación
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

VisitaDiagnostica (Persistente)				
Descripción: Representa una visita de evaluación diagnóstica realizada por un profesional. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	adultoMayor	AdultoMayor	Objeto de tipo AdultoMayor	
-	estado	estadosVisita	PENDIENTE, REALIZADA, CANCELADA	
-	fechaHora	DateTime	Fecha y hora válidas	
-	id	int	Entero positivo	
-	profesional	Usuario	Objeto de tipo Usuario	
-	resultado	String	Texto libre	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	VisitaDiagnostica()	-	void	Constructor de la clase
+	registrarResultado	()	void	Registra el resultado de la visita
+	reprogramar	()	void	Reprograma la visita diagnóstica
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

HorarioEnfermeria (Persistente)				
Descripción: Representa el turno de enfermería asignado a un usuario en un hogar para una fecha específica. Clase persistente.				
Capa de atributos				
Vis	**Nombre**	**Tipo**	**Dominio de valores**	
-	fecha	Date	Fecha válida	
-	hogar	Hogar	Objeto de tipo Hogar	
-	id	int	Entero positivo	
-	personal	Usuario	Objeto de tipo Usuario	
-	turno	turnosEnfermeria	MAÑANA, TARDE, NOCHE	
Capa de métodos				
Vis	**Nombre**	**Par. Tipos**	**Tipo Retorno**	**Intencionalidad**
+	HorarioEnfermeria()	-	void	Constructor de la clase
+	asignarPersonal	()	void	Asigna el personal de enfermería
+	cambiarTurno	()	void	Modifica el turno asignado
+	setNombreAtributo	(:Tipo Atributo)	void	Setter genérico para cada atributo
+	getNombreAtributo	()	Tipo Atributo	Getter genérico para cada atributo

4. MODELADO DE PERSISTENCIA

4.1. Patrones de Fuente de Datos

El sistema desarrollado adopta el patrón **Domain Model**, en el cual la lógica de negocio se encuentra encapsulada dentro de clases del dominio que representan entidades del contexto gerontológico, tales como *AdultoMayor*, *PlanMedico*, *DiagnosticoPsicosocial* y *Hogar*. Estas clases contienen tanto datos como comportamiento, permitiendo modelar reglas del negocio de manera coherente.

Para gestionar su persistencia en la base de datos, se implementa el patrón **Data Mapper**, el cual introduce una capa intermedia encargada de desacoplar el modelo de dominio de la base de datos. En este enfoque, los objetos del dominio no contienen lógica de acceso a datos, delegando esta responsabilidad a clases especializadas denominadas mappers.

Se selecciona este patrón debido a que permite desacoplar la lógica de negocio de la persistencia de datos, facilitando la escalabilidad, mantenibilidad y capacidad de prueba del sistema, al evitar dependencias directas entre las entidades del dominio y la base de datos.

En este enfoque, los objetos del dominio no contienen lógica de acceso a datos, delegando esta responsabilidad a clases especializadas denominadas mappers.

Adicionalmente, se emplea una estrategia híbrida que combina características de los patrones **Row Data Gateway** y **Table Data Gateway**. Cada mapper actúa como un punto de acceso a los datos de una tabla específica (Table Data Gateway), permitiendo ejecutar operaciones sobre conjuntos de registros, mientras que también permite trabajar con instancias individuales del dominio (Row Data).

Este enfoque permite:

- Separar completamente la lógica de negocio de la persistencia.
- Centralizar el acceso a datos en componentes especializados.
- Facilitar el mantenimiento y evolución del sistema.
- Permitir cambios en la base de datos sin afectar el dominio.

4.2. Estrategia de Mapeo

La estrategia de mapeo implementada se basa en el patrón **Data Mapper**, donde cada clase persistente del dominio tiene un mapper específico encargado de gestionar su persistencia y recuperación de datos.

En el contexto del sistema desarrollado, este enfoque se aplica a entidades como AdultoMayor, PlanMedico, DiagnosticoPsicosocial y Hogar, donde cada una cuenta con su respectivo mapper encargado de gestionar su persistencia en la base de datos.

En esta implementación, no se utilizan métodos genéricos como `toDomain` o `toEntity`. En su lugar, cada mapper define directamente operaciones de persistencia y consulta, dentro de las cuales se realiza de manera explícita la construcción de los objetos del dominio.

Cada mapper encapsula completamente la lógica necesaria para transformar los datos provenientes de la base de datos en instancias del dominio, resolviendo también sus relaciones. Por ejemplo, al recuperar un *AdultoMayor*, el mapper construye el objeto incluyendo atributos simples, conversión de enumeraciones y referencias a otras entidades relacionadas.

La lógica de construcción del objeto se encuentra integrada dentro de los métodos de consulta, evitando la necesidad de una capa adicional de transformación o ensamblaje de objetos. Esto permite mantener el control explícito sobre cómo se materializan los objetos del dominio.

Los mappers presentan dependencias entre sí para manejar relaciones del dominio. Por ejemplo:

- *PlanMedicoMapper* depende de *MedicamentoMapper* y *TerapiaMapper*.
- *AdultoMayorMapper* depende de *BitacoraMapper*, *PlanMedicoMapper* y *DiagnosticoPsicosocialMapper*.

Estas relaciones se modelan como dependencias de tipo «*use*», donde un mapper invoca a otros para completar la construcción de objetos agregados.

Este enfoque permite:

- Mantener la separación entre dominio y persistencia.
- Centralizar la lógica de acceso a datos en los mappers.
- Controlar explícitamente la construcción de los objetos del dominio.

Este enfoque asegura una separación clara de responsabilidades, donde el dominio permanece independiente de la infraestructura, permitiendo una mayor flexibilidad ante cambios en la base de datos o en la lógica del sistema.

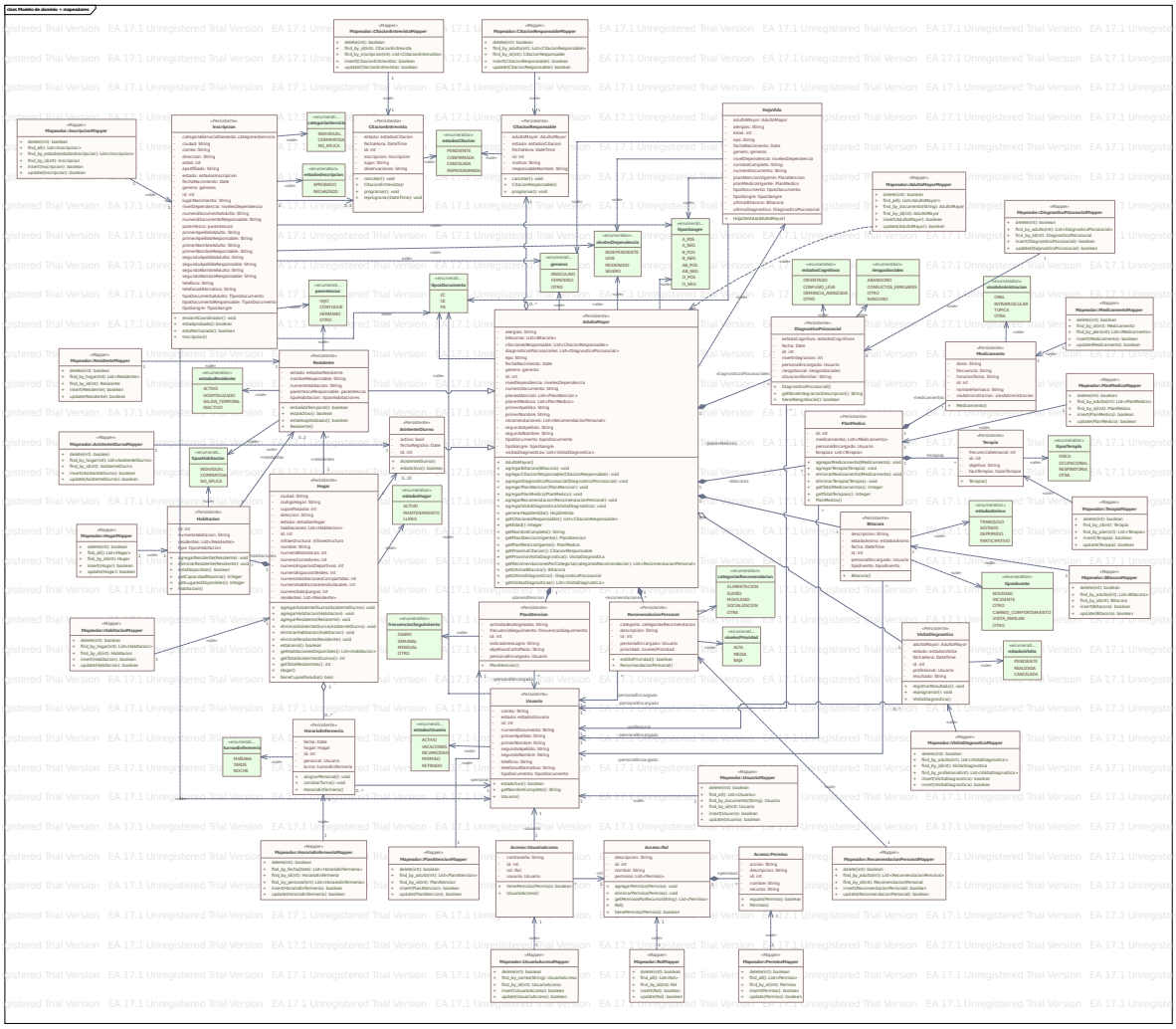


Figura 4.2: Relación entre el modelo de dominio y los mappers

4.2.1. Operaciones CRUD en los Mappers

Cada mapper implementa un conjunto definido de operaciones de persistencia, alineadas con las necesidades del sistema y con la estructura del modelo relacional.

Las operaciones implementadas son:

Insert (Insertar): Permite crear nuevos registros en la base de datos a partir de

objetos del dominio. El mapper toma los atributos del objeto y los traduce a los campos correspondientes en la tabla.

Update (Actualizar): Permite modificar registros existentes. Se actualizan únicamente los campos relevantes según los cambios realizados en el objeto del dominio.

Delete (Eliminar): Permite eliminar registros de la base de datos utilizando su identificador.

FindById: Permite recuperar una única entidad a partir de su identificador. En esta operación se construye el objeto del dominio correspondiente.

FindAll: Permite obtener una colección completa de registros de una tabla, retornando una lista de objetos del dominio.

FindByCriteria: Permite realizar consultas filtradas según uno o varios atributos (por ejemplo, por estado, por relación o por campos específicos), retornando colecciones de objetos del dominio.

En todas las operaciones de consulta, la construcción de los objetos del dominio se realiza dentro del mapper, incluyendo:

- Asignación de atributos simples.
- Conversión de valores enumerados.
- Resolución de referencias mediante claves foráneas.

Este diseño refleja una implementación controlada del patrón **Data Mapper**, combinada con características de:

- **Row Data**: manipulación de registros individuales mediante `findById`.
- **Table Data**: manejo de colecciones mediante `findAll` y `findByCriteria`.

De esta manera, los mappers actúan como el único punto de acceso a la base de datos, manteniendo encapsulada toda la lógica de persistencia y garantizando la coherencia entre el modelo de dominio y el modelo relacional.

4.3. Modelo Relacional

El modelo relacional del sistema fue diseñado de forma coherente con el modelo de dominio y la estrategia de mapeo implementada, garantizando una correspondencia

directa entre las clases persistentes y las tablas de la base de datos. Cada entidad del dominio posee su equivalente en el modelo relacional, manteniendo consistencia estructural y facilitando la implementación de los mappers.

La base de datos se organiza en cuatro bloques claramente definidos:

- **Seguridad:** Compuesto por las tablas *usuario*, *usuario_acceso*, *rol*, *permiso* y la tabla intermedia *rol_permiso*. Este módulo implementa un esquema RBAC, donde:

  - Un *usuario* tiene una única credencial en *usuario_acceso*.
  - Un *usuario_acceso* está asociado a un *rol*.
  - Un *rol* agrupa múltiples *permiso*.

  Esta separación permite desacoplar la información personal del usuario de la lógica de autenticación y autorización.

- **Dominio principal:** Centrado en la tabla *adulto_mayor*, que actúa como núcleo del sistema. A partir de esta entidad se derivan múltiples relaciones uno a muchos (1:N):

  - *diagnostico_psicosocial*
  - *plan_atencion*
  - *plan_medico*
  - *bitacora*
  - *recomendacion_personal*

  Estas relaciones reflejan directamente las agregaciones definidas en el modelo de dominio. A su vez, *plan_medico* se relaciona con *medicamento* y *terapia*, manteniendo una jerarquía de composición en la persistencia.

- **Proceso de ingreso:** Representado por la tabla *inscripcion*, la cual se mantiene desacoplada del resto del dominio, ya que corresponde a una etapa previa a la creación del *adulto_mayor*. A partir de esta entidad se derivan:

  - *citacion_entrevista*, asociada directamente a la inscripción.

  Adicionalmente, existen procesos posteriores vinculados al adulto mayor ya registrado:

  - *citacion_responsable*

- *visita_diagnostica*

Esto refleja la transición entre el proceso de ingreso y la gestión operativa del sistema.

- **Infraestructura:** Incluye las tablas *hogar*, *habitacion*, *residente*, *asistente_diurno* y *horario_enfermeria*. En este bloque:

  - Un *hogar* posee múltiples *habitacion*.
  - Un *residente* vincula un *adulto_mayor* con un *hogar* y una *habitacion*.
  - *horario_enfermeria* asigna personal (*usuario*) a un hogar en una fecha y turno específico.

  Estas relaciones reflejan la gestión física y operativa del entorno gerontológico.

Las relaciones del modelo siguen principalmente cardinalidades uno a muchos (1:N), implementadas mediante claves foráneas. En casos específicos se aplican restricciones adicionales, como:

- Unicidad en *numero_documento* de *usuario* y *adulto_mayor*.
- Unicidad en *usuario_acceso.usuario_id* para garantizar una sola credencial por usuario.
- Restricciones compuestas, como en *horario_enfermeria* (personal, fecha, turno).

El uso de enumeraciones en múltiples tablas (por ejemplo, *estadosUsuario*, *tiposEvento*, *nivelesDependencia*) asegura que los valores almacenados correspondan exactamente a los definidos en el dominio, manteniendo consistencia entre capas sin necesidad de validaciones adicionales en la base de datos.

Finalmente, el diseño relacional está alineado con la estrategia de mapeo implementada, permitiendo que cada mapper opere directamente sobre su tabla correspondiente mediante operaciones *insert*, *update*, *delete* y consultas (*findById*, *findAll*, *findByCriteria*), sin necesidad de transformaciones intermedias complejas.

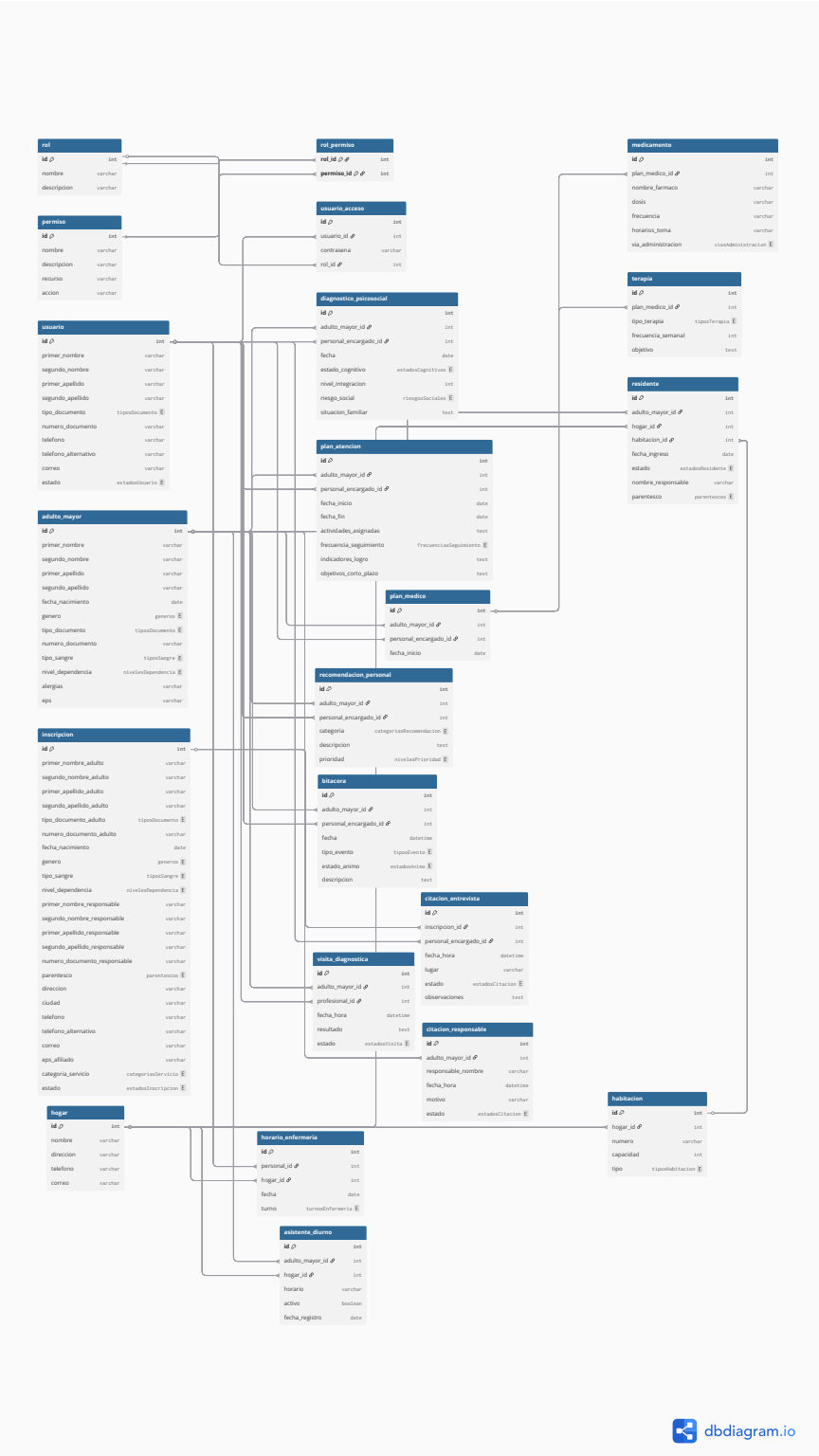


Figura 4.3: Modelo relacional del sistema (Creado con la herramienta dbdiagram.io)

Enumeraciones

Enum: `generos`

Valor	**Descripción**
MASCULINO	Género masculino
FEMENINO	Género femenino
OTRO	Otro / no especificado

Enum: `tiposDocumento`

Valor	**Descripción**
CC	Cédula de Ciudadanía
CE	Cédula de Extranjería
PA	Pasaporte
OTRO	Otro tipo de documento

Enum: `tiposSangre`

Valor	**Descripción**
A_POS	A positivo
A_NEG	A negativo
B_POS	B positivo
B_NEG	B negativo
AB_POS	AB positivo
AB_NEG	AB negativo
O_POS	O positivo
O_NEG	O negativo

Enum: `nivelesDependencia`

Valor	**Descripción**
INDEPENDIENTE	Sin necesidad de asistencia
LEVE	Apoyo mínimo u ocasional
MODERADO	Asistencia frecuente
SEVERO	Dependencia total

Enum: `estadosCognitivos`

Valor	**Descripción**
ORIENTADO	Orientado en tiempo, espacio y persona
CONFUSION_LEVE	Leve deterioro cognitivo
DEMENCIA_AVANZADA	Deterioro cognitivo severo
OTRO	No clasificado

Enum: `riesgosSociales`

Valor	**Descripción**
ABANDONO	Riesgo de abandono familiar
CONFLICTOS_FAMILIARES	Conflictos o disfunción familiar
OTRO	Otro riesgo social
NINGUNO	Sin riesgos identificados

Enum: `categoriasRecomendacion`

Valor	**Descripción**
ALIMENTACION	Nutrición y dieta
SUENO	Hábitos de sueño
MOVILIDAD	Ejercicio y movilidad
SOCIALIZACION	Integración social
OTRA	Otra categoría

Enum: `nivelesPrioridad`

Valor	**Descripción**
ALTA	Atención inmediata
MEDIA	Atención en el corto plazo
BAJA	Seguimiento rutinario

Enum: `tiposEvento`

Valor	Descripción
NOVEDAD	Situación nueva o inusual
INCIDENTE	Evento adverso o accidente
CAMBIO_COMPORTAMIENTO	Variación significativa de comportamiento
VISITA_FAMILIAR	Visita de familiar
OTRO	Otro evento

Enum: `estadosAnimo`

Valor	Descripción
TRANQUILO	Calmado y estable
AGITADO	Inquieto o ansioso
DEPRIMIDO	Tristeza o apatía
PARTICIPATIVO	Activo y sociable

Enum: `viasAdministracion`

Valor	Descripción
ORAL	Por vía oral
INTRAMUSCULAR	Inyección muscular
TOPICA	Aplicación sobre piel o mucosas
OTRA	Otra vía

Enum: `tiposTerapia`

Valor	Descripción
FISICA	Rehabilitación y movilidad
OCUPACIONAL	Actividades de la vida diaria
RESPIRATORIA	Función respiratoria
OTRA	Otro tipo

Enum: `estadosUsuario`

Valor	**Descripción**
ACTIVO	Activo en el sistema
VACACIONES	Ausente por vacaciones
INCAPACIDAD	Incapacidad médica
PERMISO	Permiso temporal
RETIRADO	Dado de baja

Enum: `frecuenciasSeguimiento`

Valor	**Descripción**
DIARIO	Cada día
SEMANAL	Cada semana
MENSUAL	Cada mes
OTRO	Frecuencia no estándar

Enum: `estadosInscripcion`

Valor	**Descripción**
PENDIENTE	En revisión
APROBADO	Aceptada
RECHAZADO	No aprobada

Enum: `categoriasServicio`

Valor	**Descripción**
INDIVIDUAL	Servicio exclusivo
COMPARTIDA	Servicio compartido
NO_APLICA	No aplica

Enum: `parentescos`

Valor	**Descripción**
HIJO	Hijo(a)
CONYUGUE	Cónyuge o pareja
HERMANO	Hermano(a)
OTRO	Otro parentesco

Enum: `estadosCitacion`

Valor	**Descripción**
PENDIENTE	Sin confirmar
CONFIRMADA	Confirmada
CANCELADA	Anulada
REPROGRAMADA	Pospuesta

Enum: `estadosVisita`

Valor	**Descripción**
PENDIENTE	Programada, no realizada
REALIZADA	Completada
CANCELADA	Anulada

Enum: `turnosEnfermeria`

Valor	**Descripción**
MANANA	Turno de la mañana
TARDE	Turno de la tarde
NOCHE	Turno nocturno

Enum: `estadosResidente`

Valor	**Descripción**
ACTIVO	Presente en el hogar
HOSPITALIZADO	En atención hospitalaria
SALIDA_TEMPORAL	Salida temporal autorizada
INACTIVO	Dado de baja

Enum: `tiposHabitacion`

Valor	**Descripción**
INDIVIDUAL	Uso exclusivo
COMPARTIDA	Dos o más residentes
NO_APLICA	No aplica

Módulo de Seguridad

usuario

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
primer_nombre	VARCHAR	NOT NULL	Primer nombre
segundo_nombre	VARCHAR	NULL	Segundo nombre
primer_apellido	VARCHAR	NOT NULL	Primer apellido
segundo_apellido	VARCHAR	NULL	Segundo apellido
tipo_documento	tiposDocumento	NOT NULL	Tipo de documento
numero_documento	VARCHAR	NOT NULL, UNIQUE	Número de documento
telefono	VARCHAR	NULL	Teléfono principal
telefono_alternativo	VARCHAR	NULL	Teléfono alternativo
correo	VARCHAR	NOT NULL, UNIQUE	Correo electrónico
estado	estadosUsuario	NOT NULL	Estado en el sistema

rol

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del rol
descripcion	VARCHAR	NULL	Descripción del rol

permiso

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del permiso
descripcion	VARCHAR	NULL	Descripción
recurso	VARCHAR	NOT NULL	Módulo al que aplica
accion	VARCHAR	NOT NULL	Acción permitida

rol_permiso

Atributo	Tipo	Restricciones	Descripción
rol_id	INT	FK → rol.id, PK	Rol asociado
permiso_id	INT	FK → permiso.id, PK	Permiso asociado

usuario_acceso

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
usuario_id	INT	FK → usuario.id, UNIQUE	Usuario propietario
contrasena	VARCHAR	NOT NULL	Contraseña hasheada
rol_id	INT	FK → rol.id	Rol asignado

Módulo Principal — Adulto Mayor

adulto_mayor

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
primer_nombre	VARCHAR	NOT NULL	Primer nombre
segundo_nombre	VARCHAR	NULL	Segundo nombre
primer_apellido	VARCHAR	NOT NULL	Primer apellido
segundo_apellido	VARCHAR	NULL	Segundo apellido
fecha_nacimiento	DATE	NOT NULL	Fecha de nacimiento
genero	generos	NOT NULL	Género
tipo_documento	tiposDocumento	NOT NULL	Tipo de documento
numero_documento	VARCHAR	NOT NULL, UNIQUE	Número de documento
tipo_sangre	tiposSangre	NOT NULL	Tipo de sangre
nivel_dependencia	nivelesDependencia	NOT NULL	Nivel de dependencia
alergias	VARCHAR	NULL	Alergias conocidas
eps	VARCHAR	NULL	EPS afiliada

diagnostico_psicosocial

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
personal_encargado_id	INT	FK → usuario.id	Profesional a cargo
fecha	DATE	NOT NULL	Fecha del diagnóstico
estado_cognitivo	estadosCognitivos	NOT NULL	Estado cognitivo
nivel_integracion	INT	NOT NULL	Nivel de integración social
riesgo_social	riesgosSociales	NOT NULL	Riesgo social identificado
situacion_familiar	TEXT	NULL	Situación familiar

plan_atencion

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único

Atributo	Tipo	Restricciones	Descripción
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
personal_encargado_id	INT	FK → usuario.id	Responsable
fecha_inicio	DATE	NOT NULL	Fecha de inicio
fecha_fin	DATE	NULL	Fecha de fin
actividades_asignadas	TEXT	NULL	Actividades asignadas
frecuencia_seguimiento	frecuenciasSeguimiento	NOT NULL	Frecuencia de seguimiento
indicadores_logro	TEXT	NULL	Indicadores de logro
objetivos_corto_plazo	TEXT	NULL	Objetivos a corto plazo

plan_medico

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
personal_encargado_id	INT	FK → usuario.id	Responsable
fecha_inicio	DATE	NOT NULL	Fecha de inicio

medicamento

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
plan_medico_id	INT	FK → plan_medico.id	Plan médico
nombre_farmaco	VARCHAR	NOT NULL	Nombre del fármaco
dosis	VARCHAR	NOT NULL	Dosis por toma
frecuencia	VARCHAR	NOT NULL	Frecuencia de administración
horarios_toma	VARCHAR	NULL	Horarios de toma
via_administracion	viasAdministracion	NOT NULL	Vía de administración

terapia

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
plan_medico_id	INT	FK → plan_medico.id	Plan médico
tipo_terapia	tiposTerapia	NOT NULL	Tipo de terapia
frecuencia_semanal	INT	NOT NULL	Sesiones por semana
objetivo	TEXT	NULL	Objetivo terapéutico

recomendacion_personal

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
personal_encargado_id	INT	FK → usuario.id	Profesional
categoria	categoriasRecomendacion	NOT NULL	Categoría
descripcion	TEXT	NOT NULL	Descripción
prioridad	nivelesPrioridad	NOT NULL	Prioridad

bitacora

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
personal_encargado_id	INT	FK → usuario.id	Personal que registra
fecha	DATETIME	NOT NULL	Fecha y hora del evento
tipo_evento	tiposEvento	NOT NULL	Tipo de evento
estado_animo	estadosAnimo	NOT NULL	Estado anímico
descripcion	TEXT	NOT NULL	Descripción del evento

Módulo de Proceso de Ingreso

inscripcion

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
primer_nombre_adulto	VARCHAR	NOT NULL	Primer nombre del adulto
segundo_nombre_adulto	VARCHAR	NULL	Segundo nombre del adulto
primer_apellido_adulto	VARCHAR	NOT NULL	Primer apellido del adulto
segundo_apellido_adulto	VARCHAR	NULL	Segundo apellido del adulto
tipo_documento_adulto	tiposDocumento	NOT NULL	Tipo de documento
numero_documento_adulto	VARCHAR	NOT NULL, INDEX	Número de documento
fecha_nacimiento	DATE	NOT NULL	Fecha de nacimiento
genero	generos	NOT NULL	Género
tipo_sangre	tiposSangre	NOT NULL	Tipo de sangre
nivel_dependencia	nivelesDependencia	NOT NULL	Nivel de dependencia
primer_nombre_responsable	VARCHAR	NOT NULL	Primer nombre del responsable
segundo_nombre_responsable	VARCHAR	NULL	Segundo nombre del responsable
primer_apellido_responsable	VARCHAR	NOT NULL	Primer apellido del responsable
segundo_apellido_responsable	VARCHAR	NULL	Segundo apellido del responsable
numero_documento_responsable	VARCHAR	NOT NULL, INDEX	Documento del responsable
parentesco	parentescos	NOT NULL	Parentesco
direccion	VARCHAR	NOT NULL	Dirección de residencia
ciudad	VARCHAR	NOT NULL	Ciudad
telefono	VARCHAR	NOT NULL	Teléfono principal
telefono_alternativo	VARCHAR	NULL	Teléfono alternativo
correo	VARCHAR	NULL	Correo electrónico
eps_afiliado	VARCHAR	NULL	EPS afiliada
categoria_servicio	categoriasServicio	NOT NULL	Modalidad del servicio
estado	estadosInscripcion	NOT NULL	Estado de la inscripción

citacion_entrevista

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
inscripcion_id	INT	FK → inscripcion.id	Inscripción asociada
personal_encargado_id	INT	FK → usuario.id	Profesional a cargo
fecha_hora	DATETIME	NOT NULL	Fecha y hora
lugar	VARCHAR	NOT NULL	Lugar
estado	estadosCitacion	NOT NULL	Estado
observaciones	TEXT	NULL	Observaciones

citacion_responsable

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
responsable_nombre	VARCHAR	NOT NULL	Nombre del responsable
fecha_hora	DATETIME	NOT NULL	Fecha y hora
motivo	VARCHAR	NOT NULL	Motivo de la citación
estado	estadosCitacion	NOT NULL	Estado

visita_diagnostica

Atributo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id	Adulto mayor
profesional_id	INT	FK → usuario.id	Profesional
fecha_hora	DATETIME	NOT NULL	Fecha y hora
resultado	TEXT	NULL	Resultados de la visita
estado	estadosVisita	NOT NULL	Estado

Módulo de Infraestructura

hogar

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
nombre	VARCHAR	NOT NULL	Nombre del hogar
direccion	VARCHAR	NOT NULL	Dirección física
telefono	VARCHAR	NULL	Teléfono
correo	VARCHAR	NULL	Correo electrónico

habitacion

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
hogar_id	INT	FK → hogar.id	Hogar al que pertenece
numero	VARCHAR	NOT NULL	Número de habitación
capacidad	INT	NOT NULL	Capacidad máxima
tipo	tiposHabitacion	NOT NULL	Tipo de habitación
—	—	UNIQUE(hogar_id, numero)	Número único por hogar

residente

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id, UNIQUE	Adulto mayor
hogar_id	INT	FK → hogar.id	Hogar
habitacion_id	INT	FK → habitacion.id	Habitación
fecha_ingreso	DATE	NOT NULL	Fecha de ingreso
estado	estadosResidente	NOT NULL	Estado
nombre_responsable	VARCHAR	NOT NULL	Nombre del responsable
parentesco	parentescos	NOT NULL	Parentesco

asistente_diurno

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
adulto_mayor_id	INT	FK → adulto_mayor.id, UNIQUE	Adulto mayor
hogar_id	INT	FK → hogar.id	Hogar
horario	VARCHAR	NOT NULL	Horario de asistencia
activo	BOOLEAN	NOT NULL	Si está activo
fecha_registro	DATE	NOT NULL	Fecha de registro

horario_enfermeria

Atributo	**Tipo**	**Restricciones**	**Descripción**
id	INT	PK, AUTO_INC	Identificador único
personal_id	INT	FK → usuario.id	Personal de enfermería
hogar_id	INT	FK → hogar.id	Hogar
fecha	DATE	NOT NULL	Fecha del turno
turno	turnosEnfermeria	NOT NULL	Turno asignado
—	—	UNIQUE(personal_id, fecha, turno)	Turno único por día

Relaciones entre Entidades

Entidad Origen	**Cardinalidad**	**Entidad Destino**	**Descripción**
usuario_acceso	N:1	usuario	Un usuario tiene un acceso
usuario_acceso	N:1	rol	Un acceso tiene un rol
rol_permiso	N:1	rol	Asocia roles con permisos
rol_permiso	N:1	permiso	Asocia permisos a roles
diagnostico_psicosocial	N:1	adulto_mayor	Diagnóstico de un adulto
diagnostico_psicosocial	N:1	usuario	Realizado por un profesional
plan_atencion	N:1	adulto_mayor	Plan de un adulto
plan_medico	N:1	adulto_mayor	Plan médico de un adulto
medicamento	N:1	plan_medico	Medicamentos del plan
terapia	N:1	plan_medico	Terapias del plan
recomendacion_personal	N:1	adulto_mayor	Recomendación para un adulto
bitacora	N:1	adulto_mayor	Eventos del adulto
citacion_entrevista	N:1	inscripcion	Citación de inscripción
citacion_responsable	N:1	adulto_mayor	Citación al responsable
visita_diagnostica	N:1	adulto_mayor	Visita domiciliaria
habitacion	N:1	hogar	Habitación de un hogar
residente	1:1	adulto_mayor	Un adulto es residente
residente	N:1	hogar	Residente en un hogar
residente	N:1	habitacion	Residente en una habitación
asistente_diurno	1:1	adulto_mayor	Adulto en modalidad diurna
horario_enfermeria	N:1	usuario	Turno de un enfermero
horario_enfermeria	N:1	hogar	Turno en un hogar

5. DISEÑO BASICO DE IGU

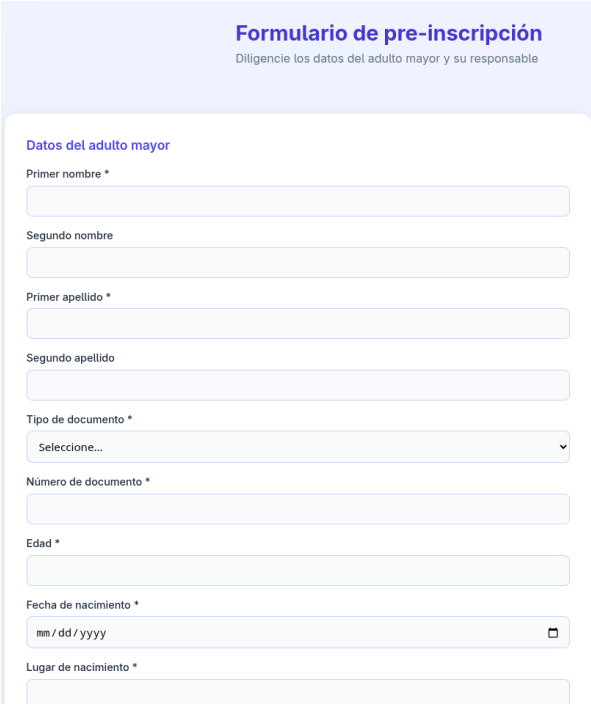
5.1. Bocetos visuales IGU

5.1.1. Acceso

Boceto para la página de acceso: contiene los botones de `Iniciar sesión` para los usuarios del sistema y el botón para diligenciar el formulario de pre-inscripción, dirigido a los responsables de adultos mayores interesados.

5.1.2. Formulario de inscripción de adultos mayores

Formulario público de pre-inscripción en el que el responsable del adulto mayor diligencia los datos personales del futuro residente y los propios. Está dividido en secciones para facilitar el ingreso de la información.



Formulario de pre-inscripción

Diligencie los datos del adulto mayor y su responsable

Datos del adulto mayor

Primer nombre *

Segundo nombre

Primer apellido *

Segundo apellido

Tipo de documento *

Seleccione...

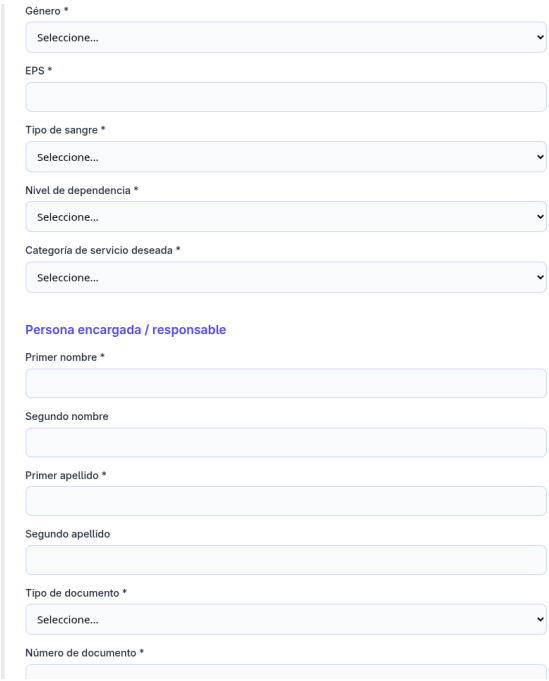
Número de documento *

Edad *

Fecha de nacimiento *

mm / dd / yyyy

Lugar de nacimiento *



Género *

Seleccione...

EPS *

Tipo de sangre *

Seleccione...

Nivel de dependencia *

Seleccione...

Categoría de servicio deseada *

Seleccione...

Persona encargada / responsable

Primer nombre *

Segundo nombre

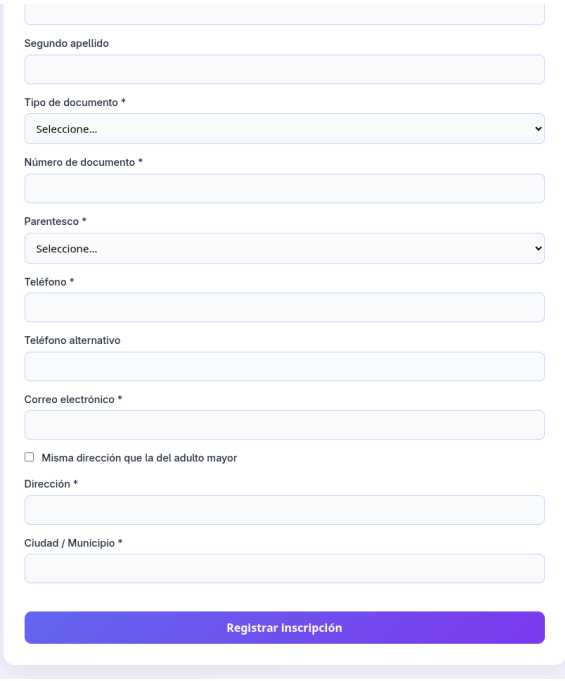
Primer apellido *

Segundo apellido

Tipo de documento *

Seleccione...

Número de documento *



Segundo apellido

Tipo de documento *

Seleccione...

Número de documento *

Parentesco *

Seleccione...

Teléfono *

Teléfono alternativo

Correo electrónico *

Misma dirección que la del adulto mayor

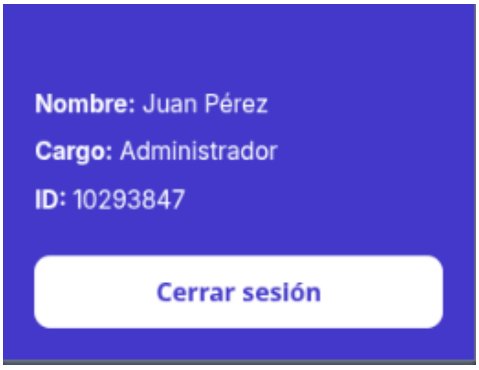
Dirección *

Ciudad / Municipio *

Registrar inscripción

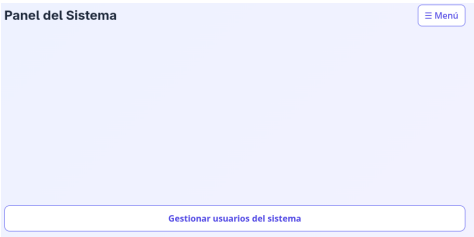
5.1.3. Cierre de sesión

A partir de este punto, es necesario indicar que todas las pantallas del sistema contienen un panel lateral desplegable que permite al usuario cerrar sesión de forma segura.



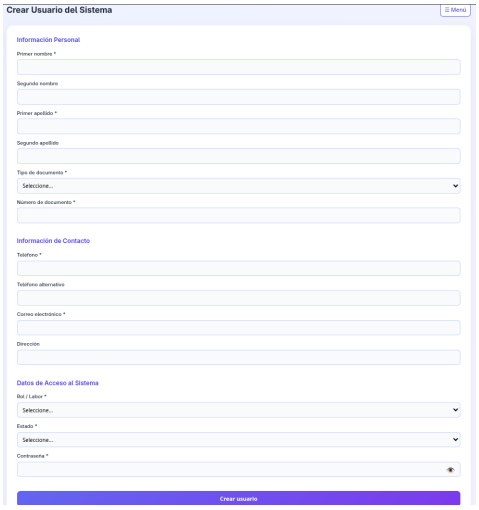
5.1.4. Administrador de usuarios

Inicio: Le presenta al administrador de usuarios la pantalla con el botón de gestionar usuarios.

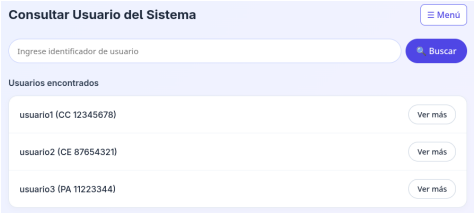


Gestionar Usuarios: Es la interfaz con todas las opciones sobre usuarios que puede realizar el administrador.

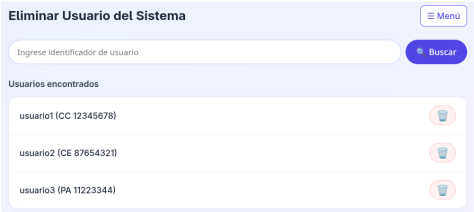
Crear Usuarios: Es el formulario con los campos de información necesarios para crear un usuario en el sistema, y el botón de crear, para hacerlo.



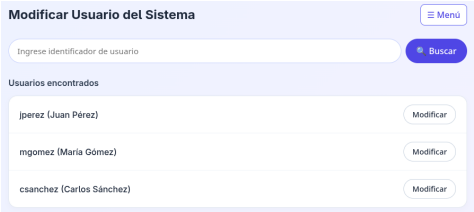
Consultar Usuarios: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para ver más información.



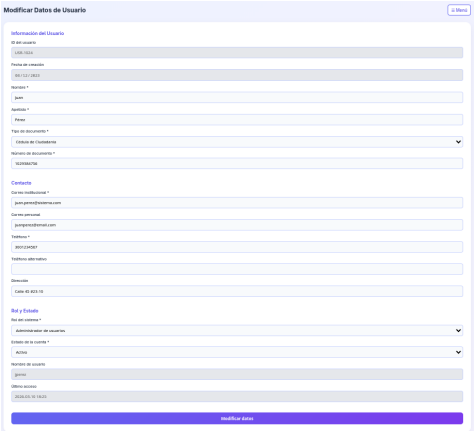
Eliminar usuarios: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para eliminar el usuario que se quiera.



Modificar Usuarios (entrada): Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para modificar el usuario que se quiera.



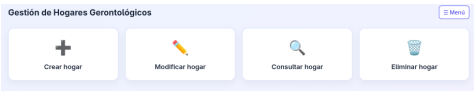
Modificar Usuarios (implementación): Muestra el formulario que permite modificar los datos del usuario previamente seleccionado.



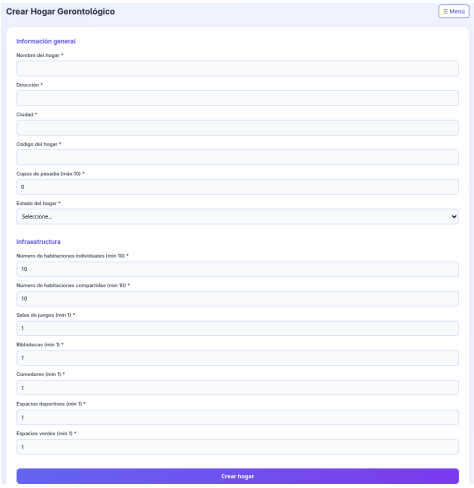
5.1.5. Administrador de Hogares

Inicio: Le presenta al administrador de hogares la pantalla con el botón de gestionar hogares.

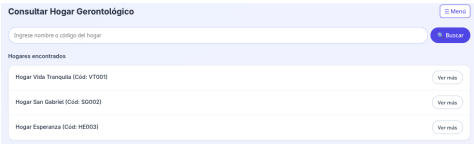
Gestionar Hogares: Es la interfaz con todas las opciones sobre hogares que puede realizar el administrador.



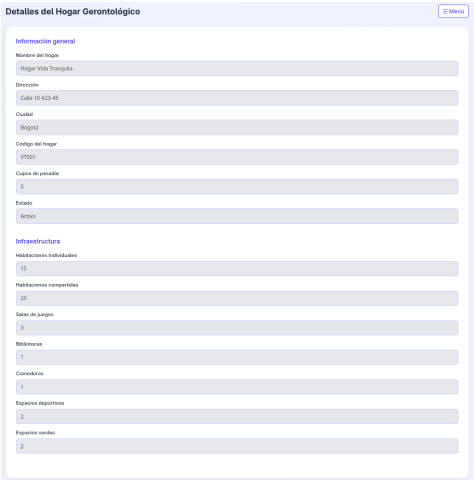
Crear Hogares: Es el formulario con los campos de información necesarios para crear un hogar en el sistema, y el botón de crear, para hacerlo.



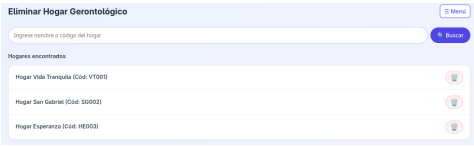
Consultar Hogares: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para ver más información.



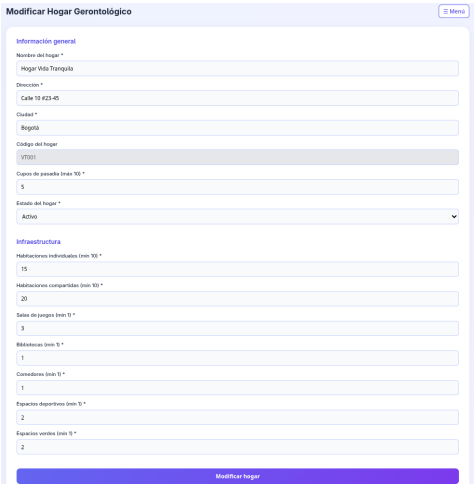
Ver más: Muestra la información del hogar seleccionado previamente.



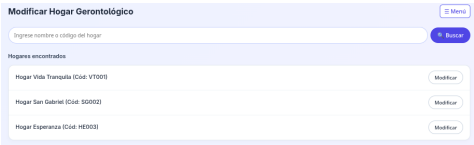
Eliminar Hogares: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para eliminar el hogar que se quiera.



Modificar Hogares (entrada): Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para modificar el hogar que se quiera.



Modificar Hogares (implementación): Muestra el formulario que permite modificar los datos del hogar previamente seleccionado.

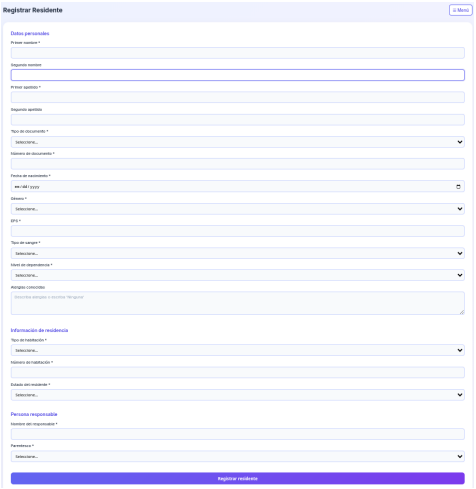


5.1.6. Administrador de Residentes

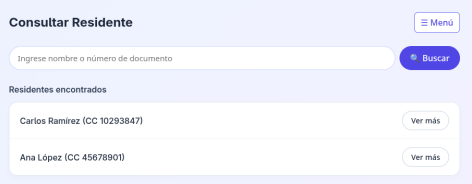
Inicio: Le presenta al administrador de residentes la pantalla con el botón de gestionar residentes.

Gestionar Residentes: Es la interfaz con todas las opciones sobre residentes que puede realizar el administrador.

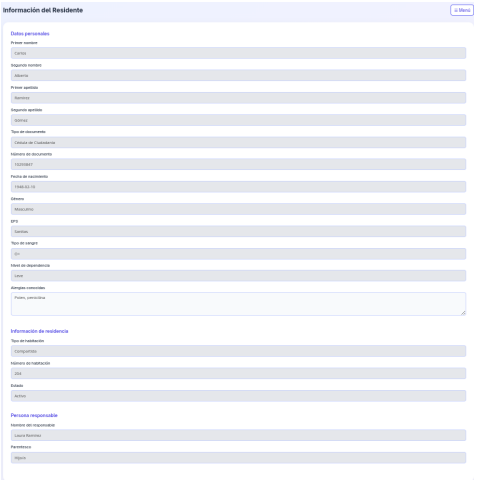
Crear Residente: Es el formulario con los campos de información necesarios para crear un residente en el sistema, y el botón de crear, para hacerlo.



Consultar Residentes: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para ver más información.



Ver más: Muestra la información del residente seleccionado previamente.



Eliminar Residentes: Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para eliminar el residente que se quiera.

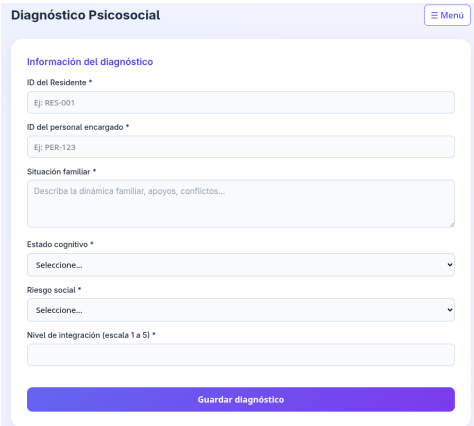
Modificar Residentes (entrada): Presenta una barra de búsqueda, una lista con los resultados de esa búsqueda, y un botón en cada resultado para modificar el residente que se quiera.

Modificar Residentes (implementación): Muestra el formulario que permite modificar los datos del residente previamente seleccionado.

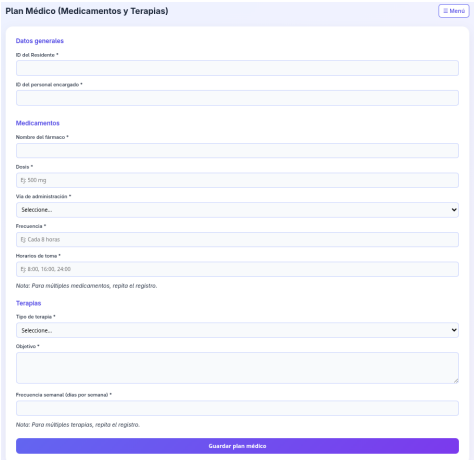
5.1.7. Formularios de seguimiento y atención al residente

A continuación se presentan los formularios utilizados por el personal del hogar para registrar la evolución diaria, planes de cuidado y recomendaciones específicas de cada residente.

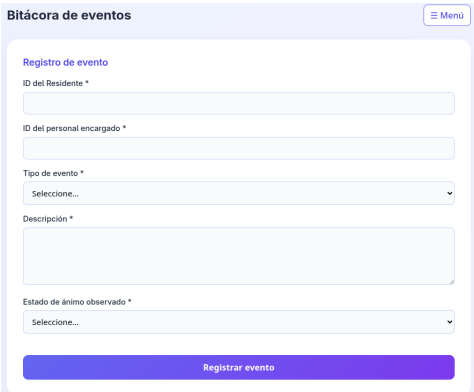
Diagnóstico Psicosocial: Permite al trabajador psicosocial registrar la situación familiar, el estado cognitivo, los riesgos sociales y el nivel de integración del residente en una escala de 1 a 5.



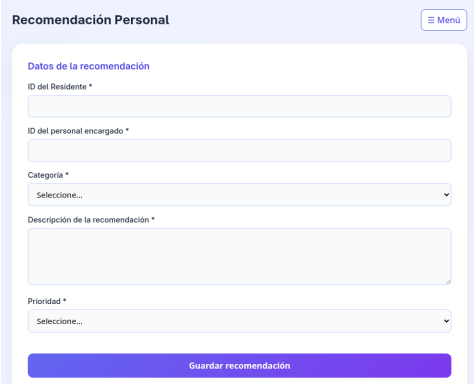
Plan Médico: Formulario destinado al personal de enfermería para consignar los medicamentos (nombre, dosis, vía, frecuencia y horarios) y las terapias (tipo, objetivo y frecuencia semanal) prescritas al residente.



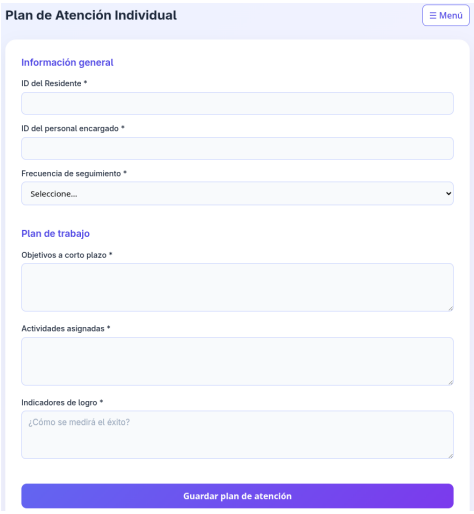
Bitácora de eventos: Registro diario de novedades, incidentes o cambios relevantes en el comportamiento del residente. Incluye el tipo de evento, una descripción detallada y el estado de ánimo observado.



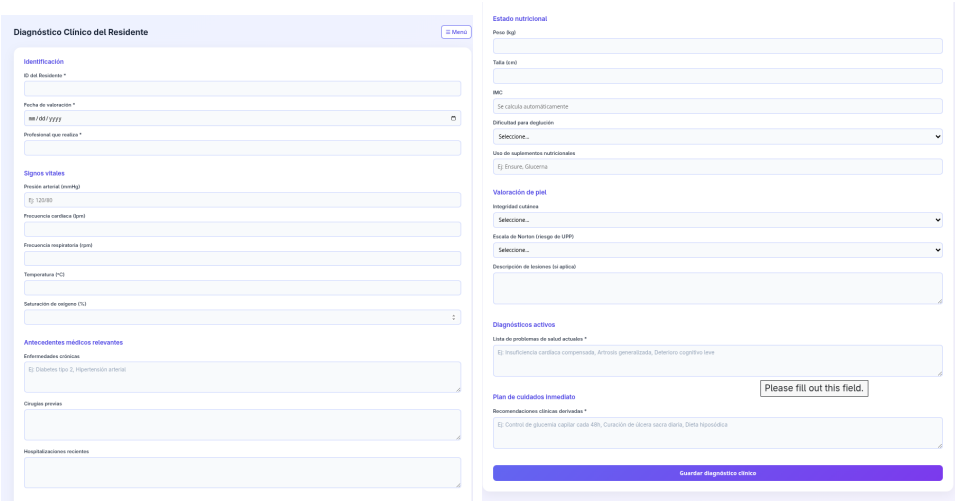
Recomendación Personal: Permite al personal encargado sugerir acciones específicas en áreas como alimentación, sueño, movilidad o socialización, asignándoles un nivel de prioridad (alta, media o baja).



Plan de Atención: Documento donde se definen los objetivos a corto plazo, las actividades asignadas y los indicadores de logro para el seguimiento integral del residente, con una frecuencia de revisión establecida.



Diagnóstico Clínico: Formulario destinado al personal de enfermería para realizar la valoración inicial o periódica del estado de salud físico y funcional del residente. Incluye signos vitales, antecedentes médicos, valoración funcional (Barthel, riesgo de caídas), estado nutricional, integridad de la piel (escala Norton), diagnósticos activos y un plan de cuidados inmediato.



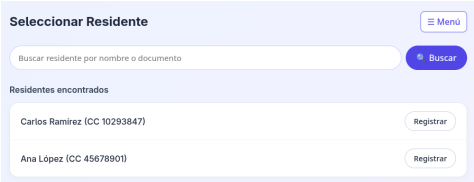
5.1.8. Panel del Trabajador Social

Esta sección presenta las interfaces destinadas al personal del área psicosocial, encargado de registrar el seguimiento diario de los residentes, elaborar planes de atención y generar reportes consolidados.

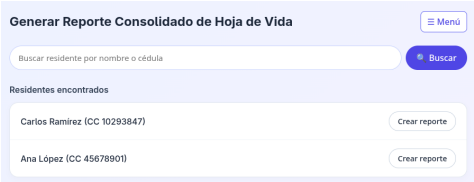
Inicio del trabajador social: Pantalla principal que ofrece dos opciones fundamentales: generar un reporte consolidado de la hoja de vida de un residente o registrar nueva información de seguimiento.

Menú de registro de información: Al seleccionar `Registrar información`, se despliega un submenú con los cuatro tipos de registro que puede realizar el trabajador social: plan de atención, diagnóstico psicosocial, bitácora diaria y recomendaciones personales.

Búsqueda de residente para registro: Una vez elegido el tipo de registro, el sistema presenta un buscador para localizar al residente sobre el cual se va a consignar la información. Cada resultado incluye un botón `Registrar` que conduce al formulario correspondiente.



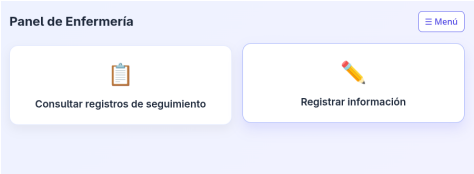
Generación de reporte consolidado: Desde la opción `Generar reporte consolidado de hoja de vida` se accede a un buscador similar. Al seleccionar un residente y presionar `Crear reporte`, el sistema genera un documento que integra toda la información relevante del adulto mayor.



5.1.9. Panel del Personal de Enfermería

Esta sección presenta las interfaces destinadas al personal de enfermería, encargado del registro diario de asistencia, la administración de medicamentos y terapias, y el seguimiento clínico de los residentes.

Inicio del personal de enfermería: Pantalla principal con acceso a la consulta de registros de seguimiento previos y al menú de registro de nueva información.



Consulta de registros de seguimiento: Al seleccionar la opción correspondiente en el panel principal, se presenta un buscador que permite localizar un residente por nombre o número de documento. Cada resultado incluye un botón `Ver registros` que conduce al historial completo de informes de seguimiento del adulto mayor.

Menú de registro de información: Despliega las distintas opciones de registro que puede realizar el personal de enfermería: recomendaciones particulares, diagnóstico clínico, plan de atención, bitácora diaria, asistencia de residentes diurnos y plan de medicamentos y terapias.

Búsqueda de residente para registro: Al seleccionar una opción de registro (excepto asistencia), se presenta un buscador para localizar al residente sobre el cual se va a consignar la información. Cada resultado incluye un botón `Registrar` que conduce al formulario específico.

Registro de asistencia de residentes diurnos: Formulario que lista a los residentes con estancia diurna, permitiendo marcar su asistencia mediante casillas de verificación. Al finalizar, se registra la asistencia del día.

5.1.10. Panel del Coordinador

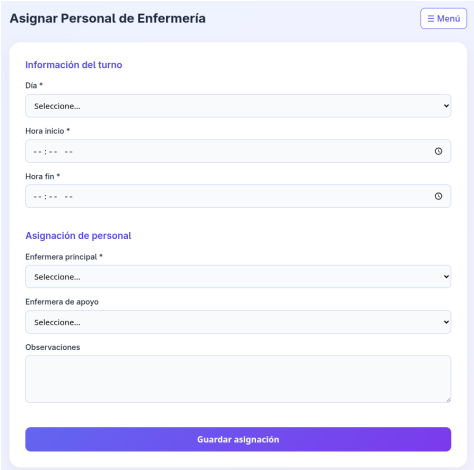
Esta sección presenta las interfaces destinadas al coordinador del hogar, quien supervisa la asignación de personal, gestiona actividades administrativas y genera reportes de seguimiento.

Inicio del coordinador: Pantalla principal que reúne las opciones de gestión propias de este rol: consultar asignación de enfermería, generar reportes consolidados, elaborar listados por categoría y ubicación, asignar personal de enfermería y gestionar actividades complementarias.

Consulta de registros de seguimiento: Interfaz que permite al coordinador buscar un residente por nombre o número de documento para acceder al historial completo de informes de seguimiento (bitácoras, recomendaciones, planes, etc.). Cada resultado incluye un botón `Ver registros` que conduce a la vista detallada.

Gestión de actividades: Submenú desde el cual el coordinador puede programar reuniones con responsables, agendar visitas diagnósticas y organizar entrevistas.

Asignación de personal de enfermería: Formulario que permite al coordinador definir los turnos diarios del personal de enfermería, indicando horarios, enfermera principal, personal de apoyo y observaciones pertinentes.

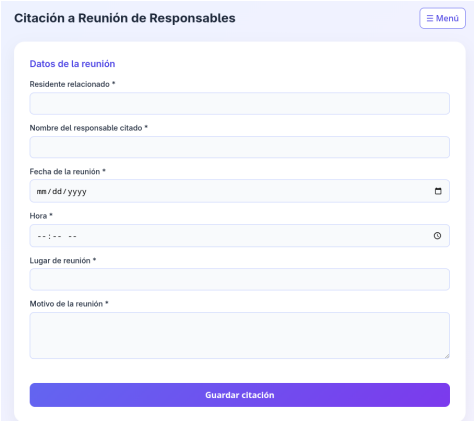


Consulta de asignación semanal: Vista tabular que muestra la distribución horaria del personal de enfermería a lo largo de la semana, facilitando la supervisión rápida de los turnos.

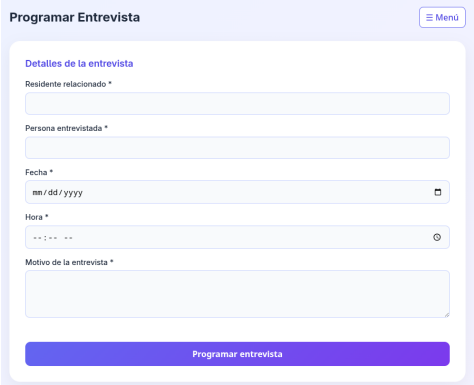
Generación de listados por categoría y ubicación: Interfaz que permite filtrar

residentes según su nivel de dependencia y ubicación dentro del hogar, para posteriormente generar un documento PDF con el listado correspondiente.

Citación a reunión de responsables: Formulario para convocar a los familiares o acudientes de un residente a una reunión, especificando fecha, hora, lugar y motivo del encuentro.

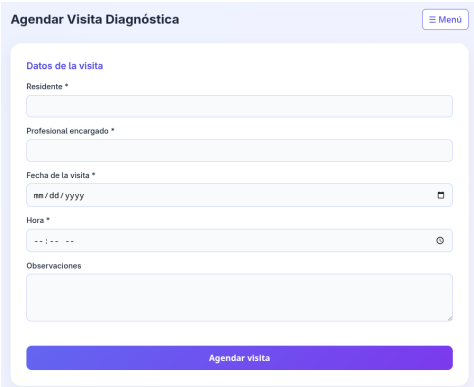


Programación de entrevista: Permite al coordinador agendar una entrevista con algún involucrado en el cuidado del residente (familiar, personal externo, etc.), registrando los datos básicos y el objetivo de la misma.



Agendamiento de visita diagnóstica: Formulario destinado a coordinar una visita de evaluación por parte de un profesional de la salud, indicando el residente, el

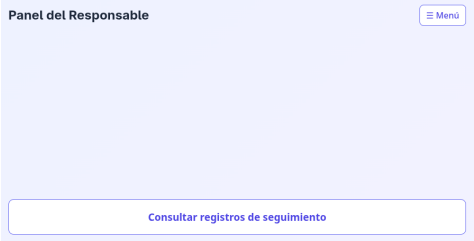
especialista encargado y los detalles de la cita.



5.1.11. Panel del Responsable del Residente

Esta sección presenta las interfaces destinadas al familiar o acudiente responsable de uno o varios adultos mayores residentes en el hogar.

Inicio del responsable: Pantalla principal que recibe al responsable tras iniciar sesión. Desde aquí puede acceder a la consulta de los registros de seguimiento de los residentes que tiene asociados.



Lista de residentes asociados: Una vez que el responsable selecciona la opción de consulta, se le presenta un listado con los nombres de los adultos mayores vinculados a su cuenta. Cada entrada dispone de un botón `Ver` que permite acceder al detalle de los registros de seguimiento del residente seleccionado.

5.1.12. Registros de seguimiento

Vista que muestra el historial de informes realizados por el personal del hogar sobre un residente. Cada registro incluye el nombre del encargado, su rol, la fecha, la hora y una descripción detallada del evento o novedad observada durante la jornada.

5.2. Mapa de navegación

El mapa de navegación presentado ilustra la estructura general de la interfaz del sistema y la relación entre las distintas páginas que lo componen. Cada uno de los elementos representados como bloques o nodos dentro del diagrama corresponde directamente a una página o vista del sistema.

Es importante destacar que los nombres incluidos en cada uno de estos bloques coinciden con las páginas definidas y descritas en la sección anterior, manteniendo así consistencia entre el diseño conceptual de la interfaz y su representación navegacional. Asimismo, cabe aclarar que la opción de cerrar sesión puede ser accedida en cualquier momento a través del panel lateral una vez el usuario ha iniciado sesión; sin embargo, en el diagrama se ha ubicado inmediatamente después del flujo principal por razones de claridad y practicidad en la representación.

De esta manera, el mapa no introduce nuevas entidades, sino que organiza visualmente las páginas ya identificadas, mostrando las posibles rutas de navegación entre ellas y facilitando la comprensión del flujo de interacción del usuario dentro del sistema.

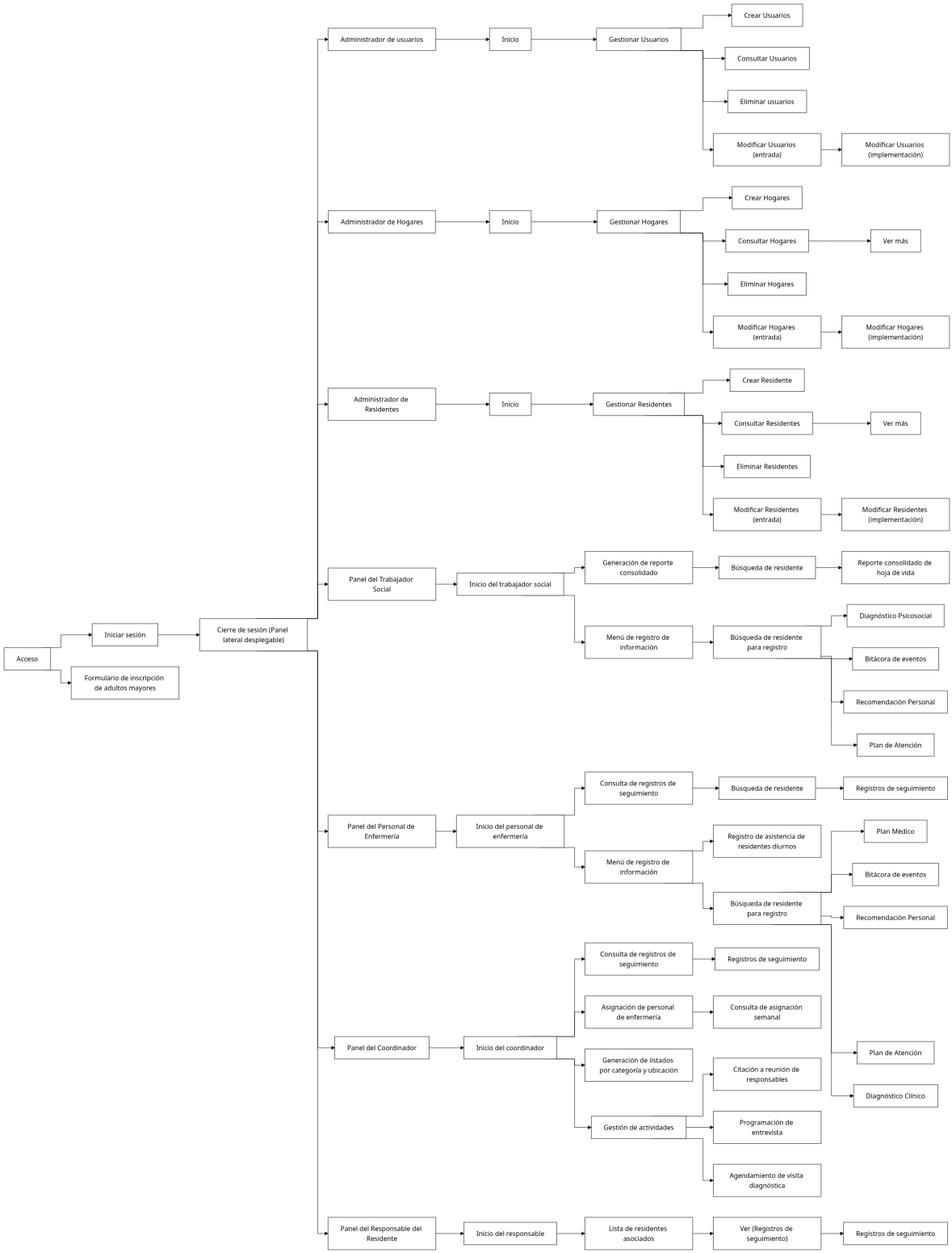


Figura 5.4: Mapa de navegación del sistema, que representa la estructura y flujo entre las páginas definidas en la interfaz de usuario

6. ULTIMA PROPUESTA DE NEGOCIACIÓN

A continuación se detallan las funcionalidades esenciales que componenen la negociación en este entregable:

- **Autenticación y Control de Acceso (CU-02):** Implementación del inicio de sesión mediante validación de credenciales (usuario y contraseña). Este proceso incluye la verificación de roles para garantizar que solo usuarios autorizados accedan a la información.

- **Cierre de Sesión (CU-53):** Funcionalidad para finalizar la sesión de manera segura, protegiendo los datos ante accesos no autorizados en ausencia del usuario.

- **Inscripción de Aspirantes (CU-04 y CU-05):** Formulario de preinscripción para que los responsables realicen solicitudes de ingreso al hogar, proceso que no requiere autenticación previa.

- **Gestión de Usuarios (CU-08 al CU-12):** Operaciones CRUD (crear, consultar, modificar y eliminar) para la administración de personal.

- **Gestión de Hogares Gerontológicos (CU-13 al CU-17):** Operaciones CRUD para la administración de los hogares y la configuración de su infraestructura institucional.

- **Gestión de Adultos Mayores (CU-48 al CU-52):** Operaciones CRUD para el registro de residentes, incluyendo su información personal, categoría de servicio y datos de sus responsables.

Componente	**Casos de Uso**	**Valor del CU**
Acceso y Seguridad	CU-02, CU-53	Control de acceso al sistema y cierre seguro de sesión.
Preinscripción	CU-04, CU-05	Canal digital para solicitudes de ingreso.
Gestión de Usuarios	CU-08 al CU-12	Administración de usuarios del sistema, con su informacion.
Gestión de Hogares	CU-13 al CU-17	Administración de los hogares, e informacion sobre su infraestructura.
Gestión de Residentes	CU-48 al CU-52	Administración de adultos mayores y su información personal.

7. REFERENCIAS

HOLISTICS SOFTWARE. dbdiagram.io: Database Relationship Diagrams Design Tool [en línea]. 2026. [Consultado el 17 de abril de 2026]. Disponible en: `https://dbdiagram.io/`

DIOSA, Henry Alberto. Consideraciones sobre la arquitectura de aplicaciones empresariales [en línea]. Bogotá: Universidad Distrital Francisco José de Caldas, [consultado el 17 de abril de 2026]. Disponible en: Material de clase de Fundamentos de Ingeniería de Software.

DIOSA, Henry Alberto. Análisis y Diseño Orientado a Objetos con UML: Modelado Estático o Estructural [en línea]. Bogotá: Universidad Distrital Francisco José de Caldas, [consultado el 17 de abril de 2026]. Disponible en: Material de clase de Fundamentos de Ingeniería de Software

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2003. 533 p.

OBJECT MANAGEMENT GROUP (OMG). Unified Modeling Language (OMG UML). Version 2.5. Needham, MA: OMG, 2015. 754 p.